

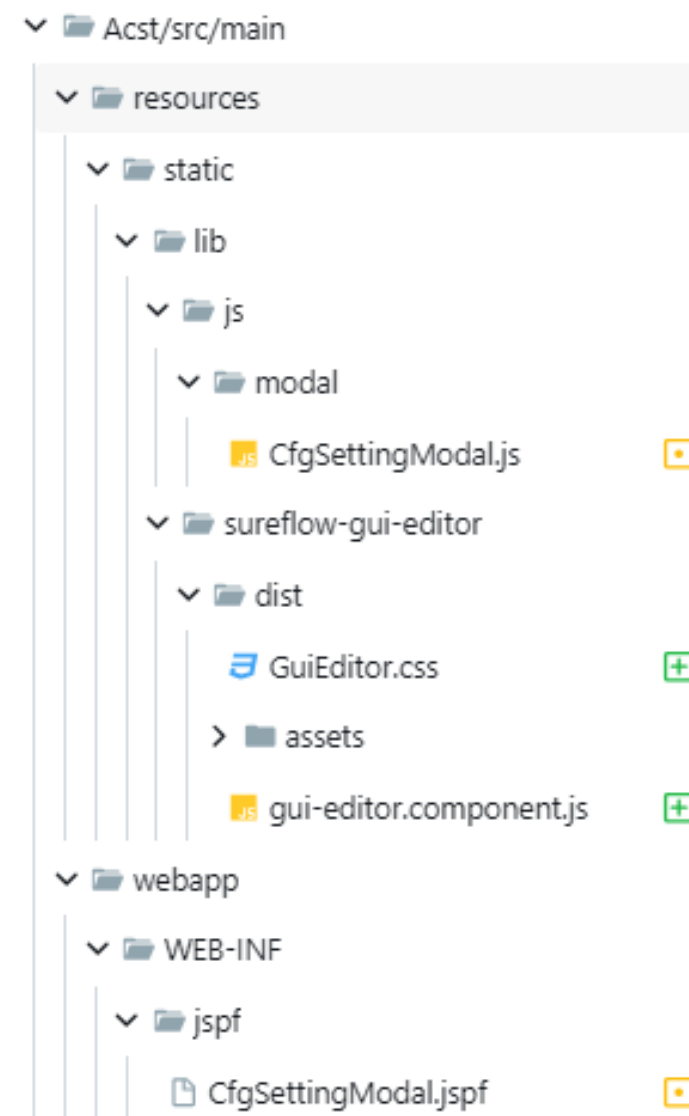
SureFlow

기능 확장 및 리팩토링

01 Sequence Diagram (Dynamic)

SURESOFT

시퀀스 다이어그램 임베드



FlowChart 임베드

Sequence Diagram도 동일하게 임베드 !

Acst\src\main\webapp\WEB-INF\jspf\SadDiagramSettingModal.jspf

Acst\src\main\resources\static\lib\js\modal\SadDiagramSettingModal.js

SAD 탭의 다이어그램 셋팅 모달 관련 코드 2개만 수정해서 임베드

Acst\src\main\resources\static\lib\sureflow-gui-editor\dist\gui-editor.component.js

Acst\src\main\resources\static\lib\sureflow-gui-editor\dist\GuiEditor.css

SureFlow 라이브러리는 Flowchart에서도 그랬듯 2개의 빌드 파일을 로드해서 사용

01 Sequence Diagram (Dynamic)

시퀀스 다이어그램 임베드

Acst\src\main\webapp\WEB-INF\jspf\SadDiagramSettingModal.jspf

```
<!-- 기존 50%: Mermaid Code -->
<div class="content-layout" style="width: 50%">
  <div style="height: 30px;">
    <div style="height: 30px; display: flex; align-items: center; gap: 8px;">
      <button type="button" class="view-toggle-btn" :class="{ 'active': isSourceView }" @click="isSourceView = false">
      <button type="button" class="view-toggle-btn active">
      <spring:message code="acst.message.ui.design.flowchart.mermaid.code" />
    </div>
    <button type="button" class="view-toggle-btn" :class="{ 'active': isSourceView }" @click="isSourceView = true">
    <spring:message code="acst.message.ui.design.flowchart.source.code" />
    </button>
  </div>
  <div style="height: 600px;">
    <textarea id="mermaid-row-sad-diagram" v-model="diagram" @input="renderMermaid">
      placeholder="<spring:message code='acst.message.ui.design.flowchart.mermaid.code.placeholder' />"
      v-if="isSourceView"</textarea>
    <div id="source-data-sad-diagram" v-else>
      <pre><code v-html="escapedCode"></code></pre>
    </div>
    <div id="mermaid-row-sad-diagram" v-model="diagram">
      placeholder="<spring:message code='acst.message.ui.design.flowchart.mermaid.code.placeholder' />"</div>
    </div>
  </div>
</div>

<!-- 기존 50%: GUI Editor (hide-editor=true 로 에디터 화면 숨기고 렌더링 화면 표시) -->
<div class="content-layout" style="width: 50%">
  <div style="height: 30px; text-align: right;">
    
    
    
  </div>
  <div style="height: 600px;">
    <mermaid-full-editor
      ref="guiEditor"
      :value="diagram"
      @input="diagram = $event"
      @svg-rendered="onSvgRendered"
      :hide-editor="true"
    ></mermaid-full-editor>
  </div>
  <div id="sad-diagram-cfg-area" ref="mermaidOutput"></div>
</div>
```

수정 내용

1. SureFlow Gui Editor의 빌드 js / css 파일 로드 구문 추가
2. 기존 mermaid svg 렌더링 영역 제거
3. 그 자리에 Gui Editor 영역 임베드
4. 기존 zoom / panning / fitview 제거 ⇒ Editor 내부 기능 사용
(Flowchart 임베드 때 CfgSettingModal 수정 사항과 동일)

01 Sequence Diagram (Dynamic)

시퀀스 다이어그램 임베드

Acst\src\main\resources\static\lib\js\modal\SadDiagramSettingModal.js

수정 내용 (Flowchart 임베드 때 CfgSettingModal 수정 사항과 동일)

```
var sadDiagramSettingModalVue = new Vue({
  el: '#sadDiagramSettingModalVue',
  data: {
    showModal: false,
    designId: '',
    kind: 'COMP_STATIC',
    rowNum: 0,
    compId: '',
    featureId: '',
    functionId: '',
    onSave: null,
    diagram: '',
    isSaveFlag: false,
    isSourceView: false,
    cfgSaveLoading: false,
    cfgZoom: 1.0,
    panX: 0,
    panY: 0,
    isPanning: false,
    panStart: { x: 0, y: 0 },
    _mermaidId: 0,
    isModalShown: false,
    renderedSvgHtml: null,
  },
});
```

1. Gui-Editor에서 수정된 SVG를 받기 위한 변수 renderedSvgHtml과
모달 여닫을때 쓰는 flag 변수 isModalShown 추가
+ 이제 더 이상 안쓰는 zoom / fit / panning에 관한 변수 삭제

```
watch: {
  showModal: function (newVal) {
    if (newVal) {
      $('#sadDiagramSettingModal').modal('show');
      // 모달 열림 애니메이션이 끝난 후 GUI 에디터 fitView를 호출해 다이어그램을 뷰포트에 맞춤
      $('#sadDiagramSettingModal').one('show.bs.modal', () => {
        this.isModalShown = true;
        if (this.$refs.guiEditor && this.$refs.guiEditor.fitView) {
          this.$refs.guiEditor.fitView();
        }
      });
    } else {
      $('#sadDiagramSettingModal').modal('hide');
      this.unbindPanListeners();
      this.renderedSvgHtml = null;
      this.isModalShown = false;
    }
  }
}
```

2. 모달 열려서 Gui Editor 호출 시 Fit View 맞춰주는 로직 및
renderedSvgHtml 받아서 저장하는 로직 추가
+ Gui Editor 내부의 공개 메서드 사용

```
// GUI 에디터가 Mermaid 다이어그램을 SVG로 렌더링 완료했을 때 호출되는 콜백.
// SVG를 보관해두고 저장 버튼을 활성화.
onSvgRendered: function (svgHtml) {
  this.renderedSvgHtml = svgHtml;
  this.isSaveFlag = true;
  if (this.isModalShown && this.$refs.guiEditor && this.$refs.guiEditor.fitView) {
    this.$refs.guiEditor.fitView();
  }
}
```

01 Sequence Diagram (Dynamic)

시퀀스 다이어그램 임베드

Acst\src\main\resources\static\lib\js\moda\SadDiagramSettingModal.js

수정 내용 (Flowchart 임베드 때 CfgSettingModal 수정 사항과 동일)

3. 이제는 GUI Editor 기능으로 치환돼서 사용하지 않는
Mermaid 영역 렌더링 / Zoom / Panning / Fit view 로직 제거

4. 이제는 Mermaid 다이어그램 SVG에 대하여
VSPICE 내부 DOM 객체가 아닌,
GUI Editor 내부 DOM 객체로 접근을 해야하므로,
이에 대한 로직도 수정.
+ renderedSvgHtml 문자열에서 view box 기준으로
파싱되게 로직 수정 ⇒ 이거 기반으로 png blob 저장

```
startPan: function (event) {
    this.isPanning = true;
    this.panStart = { x: event.clientX, y: event.clientY };
    document.body.style.cursor = 'grabbing';
},
onPan: function (event) {
    if (!this.isPanning) return;
    var dx = event.clientX - this.panStart.x;
    var dy = event.clientY - this.panStart.y;
    this.panStart = { x: event.clientX, y: event.clientY };
    this.panX += dx;
    this.panY += dy;
    this.applyCfgZoom();
},
endPan: function () {
    this.isPanning = false;
    document.body.style.cursor = 'default';
},
applyCfgZoom: function () {
    this.$nextTick(function () {
        if (!this.$refs.mermaidOutput) {
            return;
        }
        var svg = this.$refs.mermaidOutput;
        if (svg) {
            svg.style.transform = 'translate(' + this.panX + 'px, ' + this.panY + 'px)';
        }
    });
},
renderMermaid: function (retryCount) {
    retryCount = typeof retryCount === 'number' ? retryCount : 0;
    var self = this;
    var currentDiagram = (this.diagram || '').trim();
    this.isSaveFlag = false;
    if (!this.$refs.mermaidOutput) {
        return;
    }
    if (retryCount < 3) {
        this.$nextTick(function () {
            self.renderMermaid(retryCount + 1);
        });
    }
},
bindPanListeners: function (container) {
    if (this._panListenersBoundTarget === container) {
        return;
    }
    this.unbindPanListeners();
    var onMouseMove = function (e) { this.panX += e.movementX; this.panY += e.movementY; };
    var onMouseUp = function (e) { this.panX += e.movementX; this.panY += e.movementY; };
    container.addEventListener('mousemove', onMouseMove);
    container.addEventListener('mouseup', onMouseUp);
    container.addEventListener('mouseleave', onMouseLeave);
    this._panListenersBoundTarget = container;
    this._panListeners = { onMouseMove: onMouseMove, onMouseUp: onMouseUp, onMouseLeave: onMouseLeave };
},
unbindPanListeners: function () {
    if (this._panListenersBoundTarget && this._panListeners) {
        this._panListenersBoundTarget.removeEventListener('mousemove', this._panListeners.onMouseMove);
        this._panListenersBoundTarget.removeEventListener('mouseup', this._panListeners.onMouseUp);
        this._panListenersBoundTarget.removeEventListener('mouseleave', this._panListeners.onMouseLeave);
    }
    this._panListenersBoundTarget = null;
    this._panListeners = null;
},

```

```
this.cfgSaveLoading = true;
var svgData = (new XMLSerializer()).serializeToString(svgElement);
var svgBase64 = btoa(unescape(encodeURIComponent(svgData)));
var canvas = document.createElement('canvas');
var ctx = canvas.getContext('2d');
var svgWidth = svgElement.viewBox.baseVal.width;
var svgHeight = svgElement.viewBox.baseVal.height;
var svgElement.viewBox.baseVal.width = canvas.width;
var svgElement.viewBox.baseVal.height = canvas.height;

// GUI 에디터에서 받은 SVG 문자열을 파싱해 실제 크기를 읽고 Canvas로 PNG 변환
const parser = new DOMParser();
const svgDoc = parser.parseFromString(self.renderedSvgHtml, 'image/svg+xml');
const svgElement = svgDoc.querySelector('svg');

const viewBox = svgElement.getAttribute('viewBox');
const parts = viewBox.trim().split(/[ ,]+/);
const svgWidth = parseFloat(parts[2]);
const svgHeight = parseFloat(parts[3]);

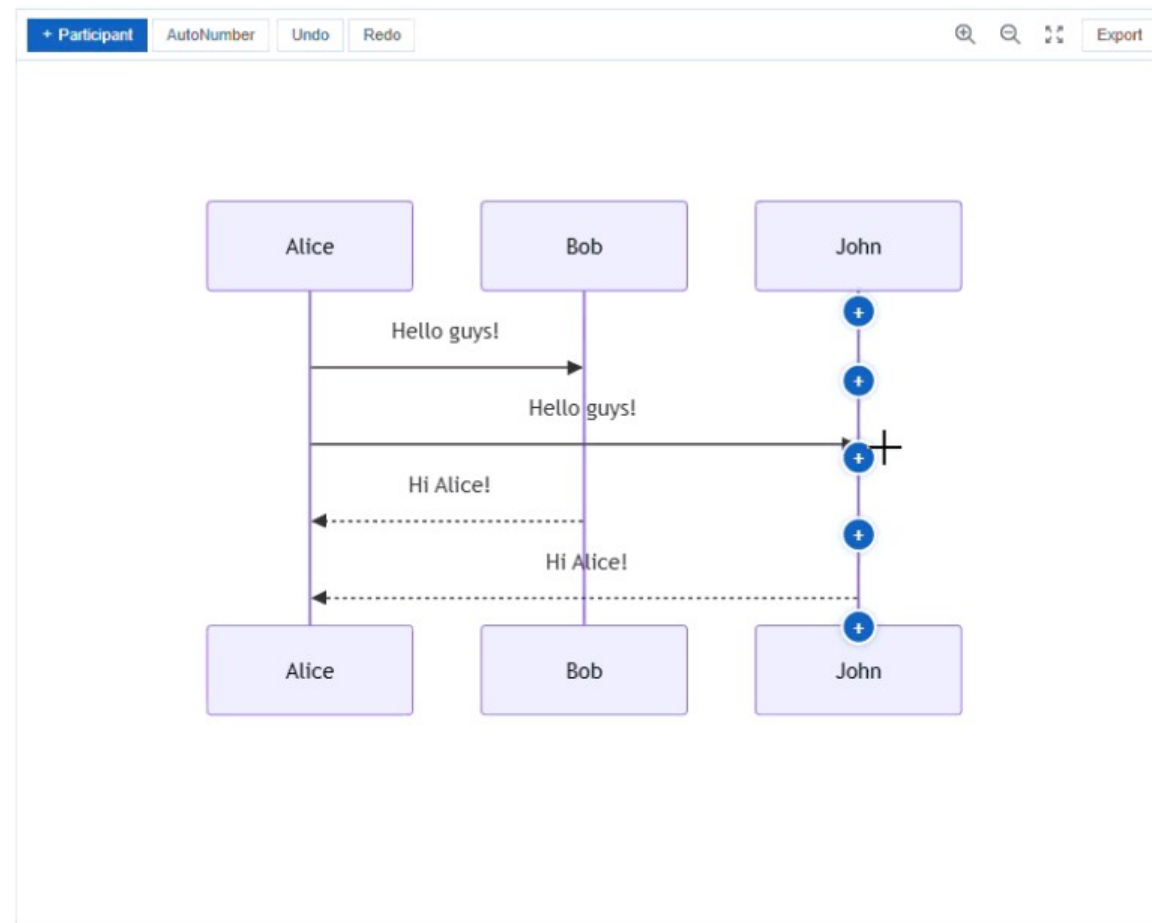
const svgData = new XMLSerializer().serializeToString(svgElement);
const svgBase64 = btoa(unescape(encodeURIComponent(svgData)));

const canvas = document.createElement('canvas');
const ctx = canvas.getContext('2d');
```


01 Sequence Diagram (Dynamic)

시퀀스 다이어그램 임베드

임베드 결과



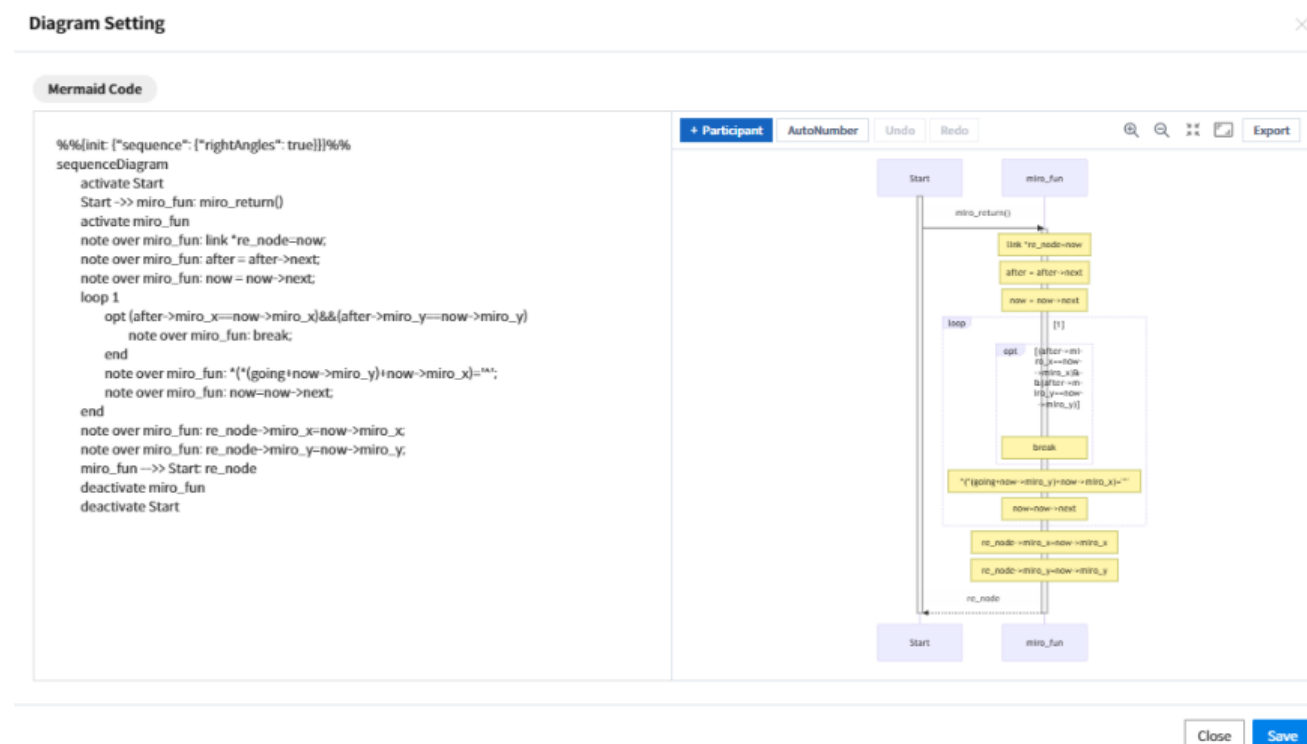
VSPICE 내에서 안정적으로 잘 작동하는 모습

- 기능 작동 이상 無
- DownLoad 기능과 Document 땀에서도 잘 적용되는것도 확인 완료
- png blob 저장 로직 수정 ⇒ 이에 따른 png 용량 변화도 없음을 확인

01 Sequence Diagram (Dynamic)

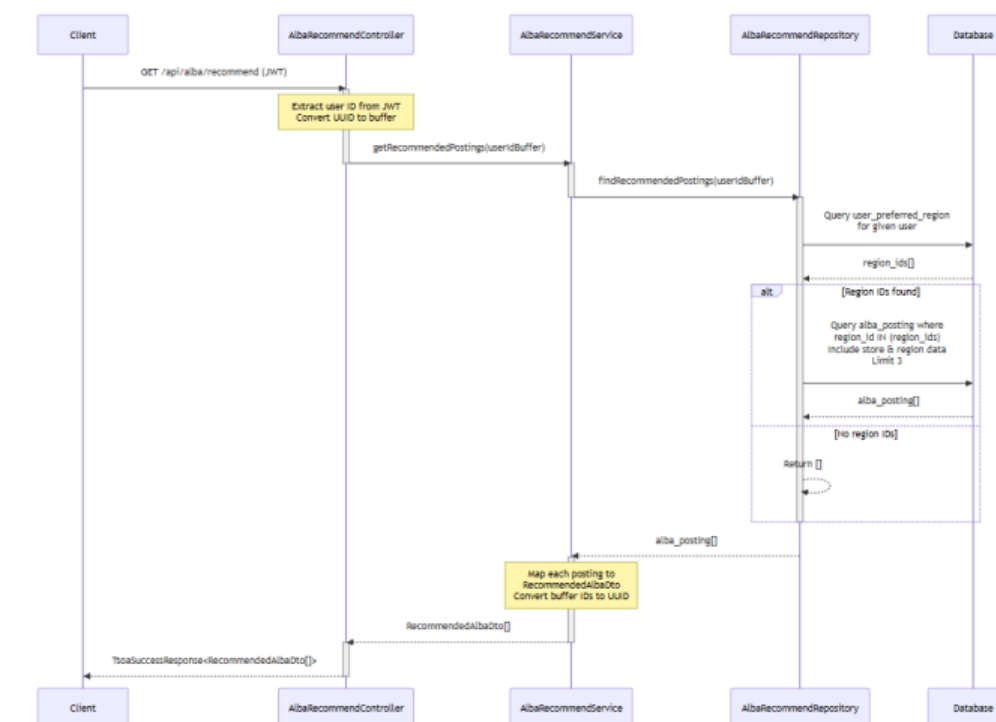
ALIRA - AI's Sequence Diagram

ALIRA AI가 생성하는 Sequnce Diagram 코드는 일반적인 Sequence Diagram과 다르다.



ALIRA AI가 생성하는 시퀀스 다이어그램

코드 실행 추적을 위해 함수 내부 문장 실행 과정을 note로 찍어 표현한 흐름도



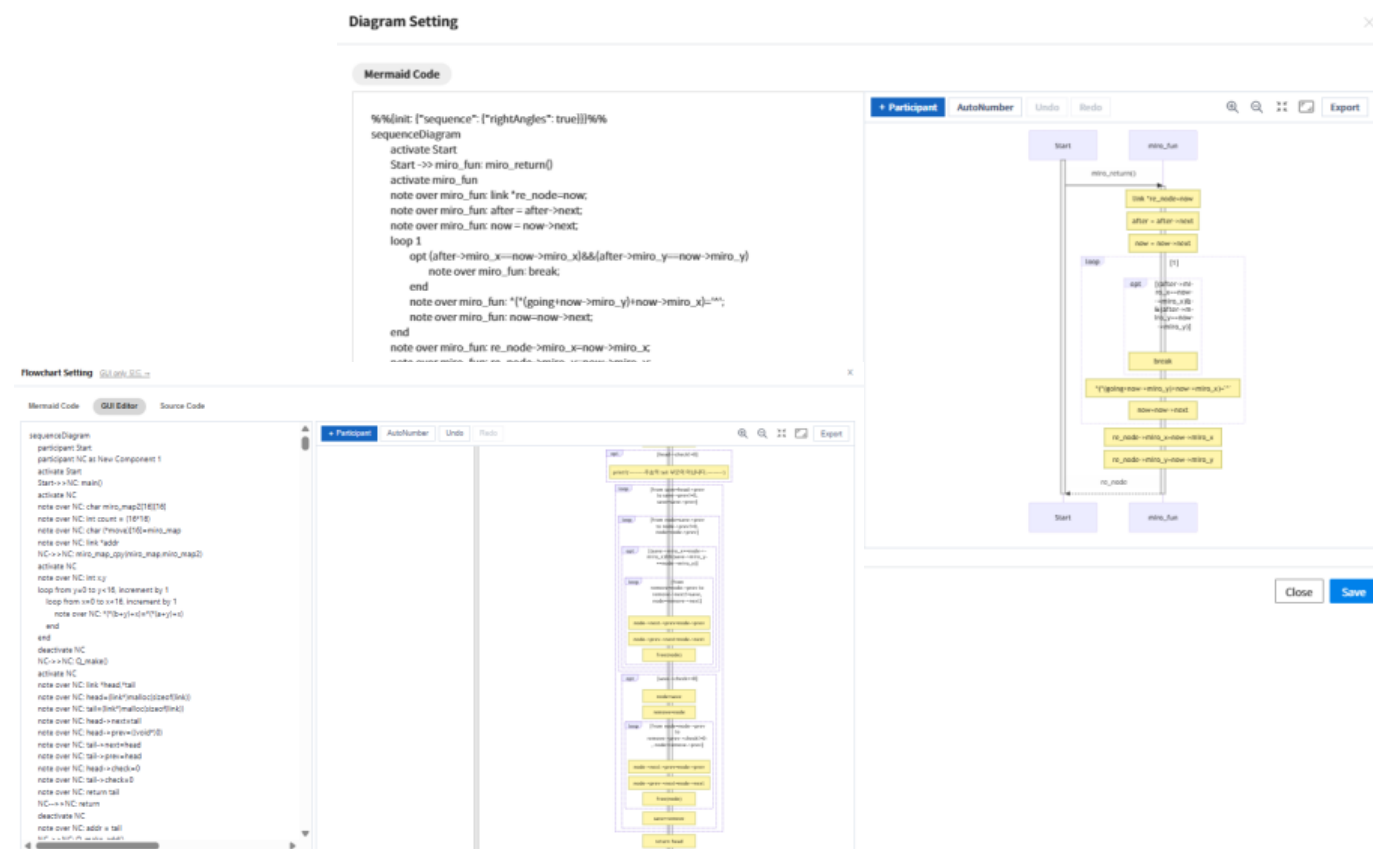
일반적인 시퀀스 다이어그램

계층/컴포넌트 상호작용을 위한 edge 위주의 시퀀스 다이어그램

01 Sequence Diagram (Dynamic)

ALIRA - AI's Sequence Diagram

ALIRA AI가 생성하는 Sequnce Diagram 코드는 일반적인 Sequence Diagram과 다르다.



ALIRA AI가 생성하는 시퀀스 다이어그램

다른 목적, 다른 용도로 일반적인 형상과 다르다.
ALIRA가 생성한 시퀀스 다이어그램이 틀렸다는건 아니다.

edge가 아닌 **note** 위주로 시퀀스 다이어그램이 구성되어있고,
box 문이 훨씬 nested된 모습.
경우에 따라서는 일반적인 시퀀스 다이어그램보다 훨씬 복잡한 형상이 된다.

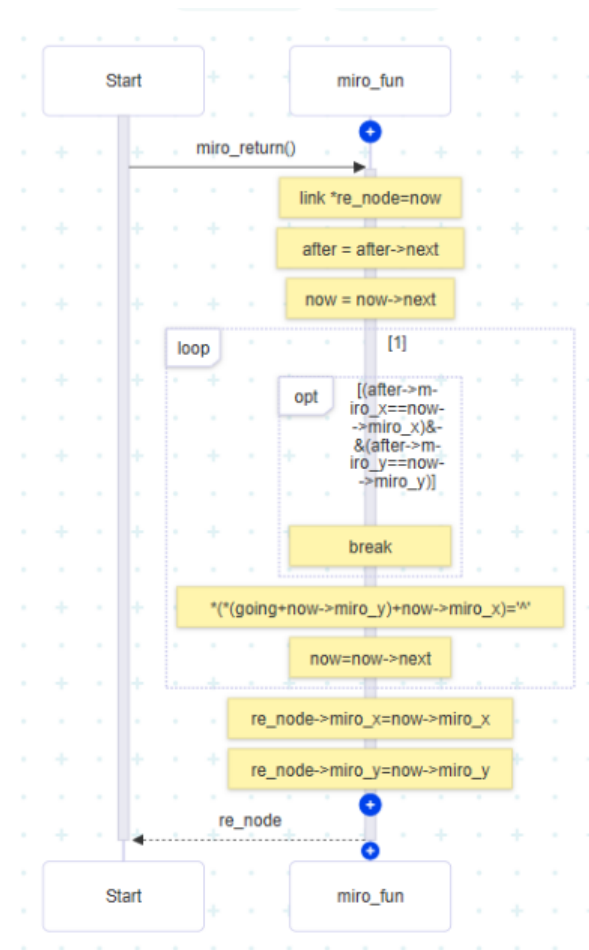
그래서 보편적인 Sequence Diagram 편집보단
더 세밀하게 편집 가능한 다른 방향이 필요하다고 생각을 했다.

01 Sequence Diagram (Dynamic)

SURESOFT

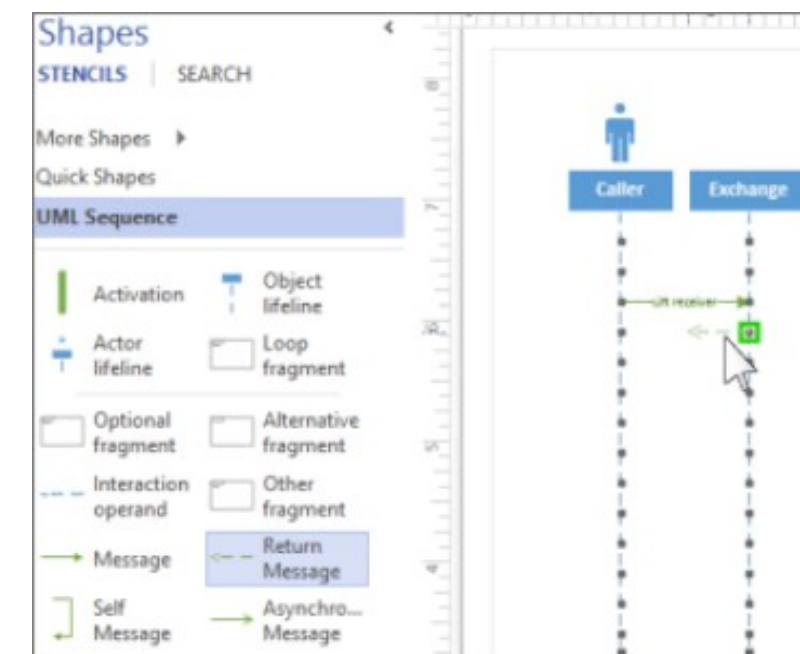
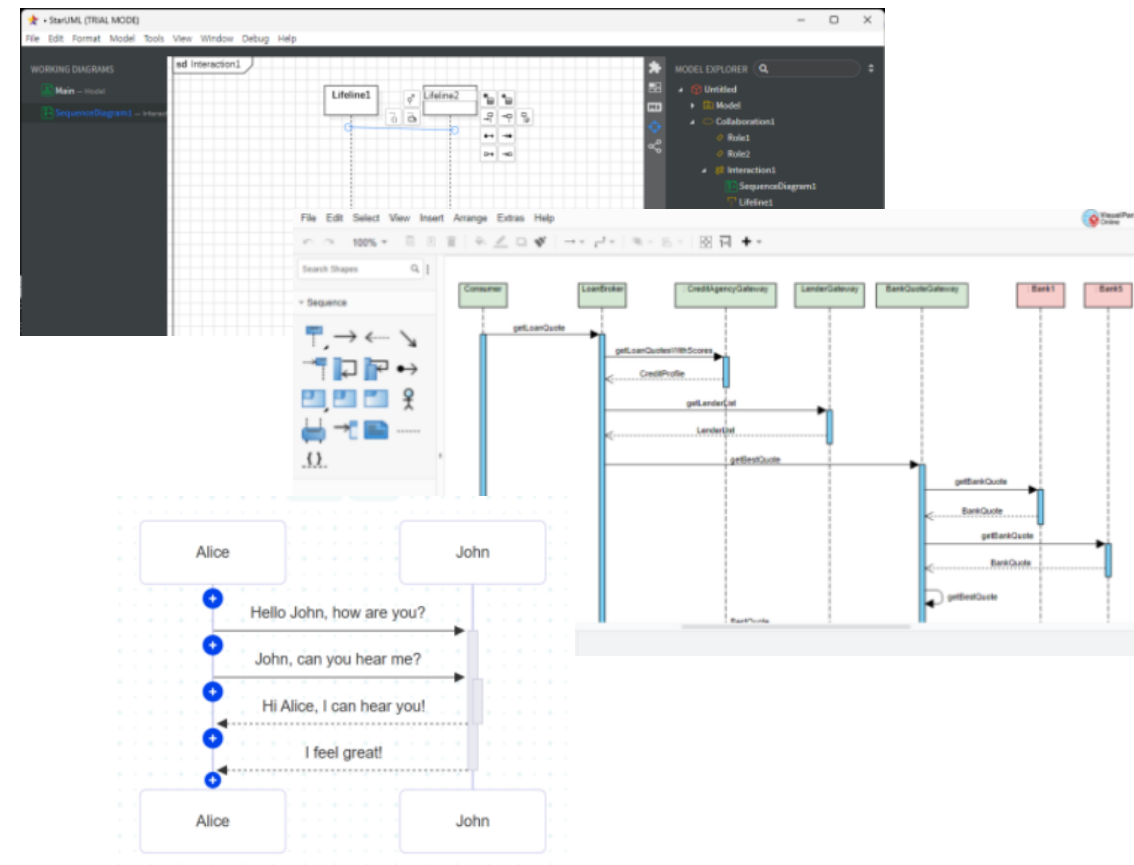
ALIRA - AI's Sequence Diagram

레퍼런스 찾기



Mermaid.ai에서의
시퀀스 다이어그램 편집

Mermaid에서는
기본적으로 Note를 편집 객체로 인식하지 않는다.
그래서 Note 사이에는 뭔가 편집을 할 수 없었다.



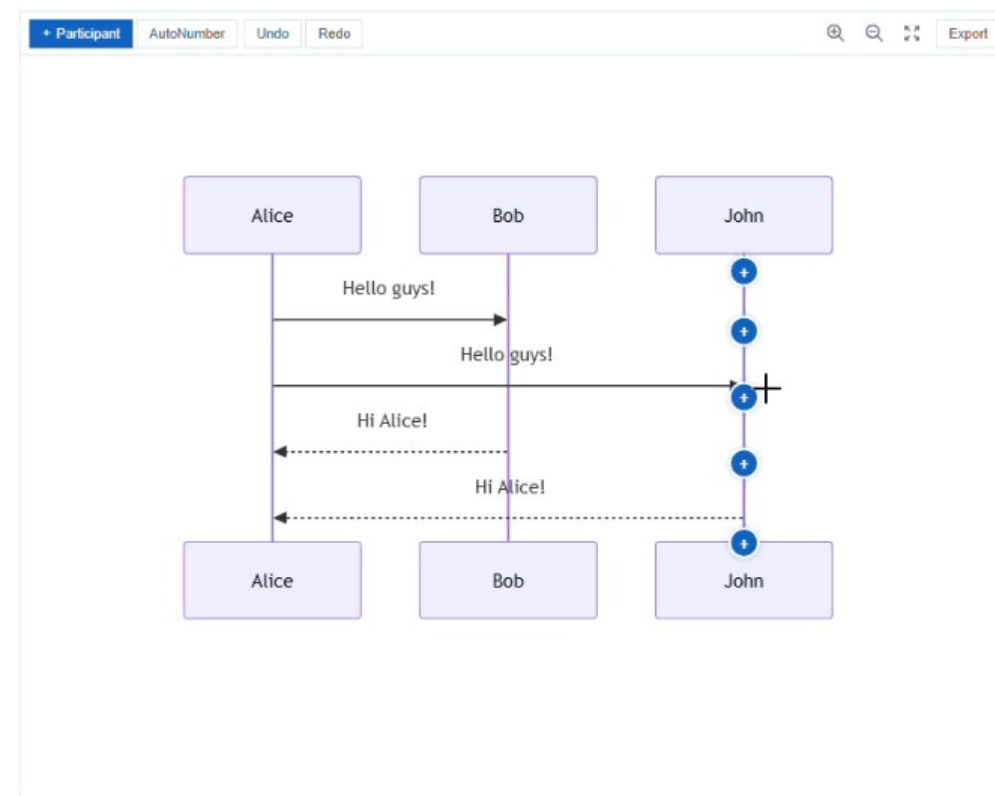
다른 레퍼런스도 찾아보았는데,
그림판처럼 그리는거나
틀이 잡혀있는 경우..
모두 정형화된 틀에서 벗어나진 않았다.

01 Sequence Diagram (Dynamic)

SURESOFT

ALIRA - AI's Sequence Diagram

없으면 만들어보자

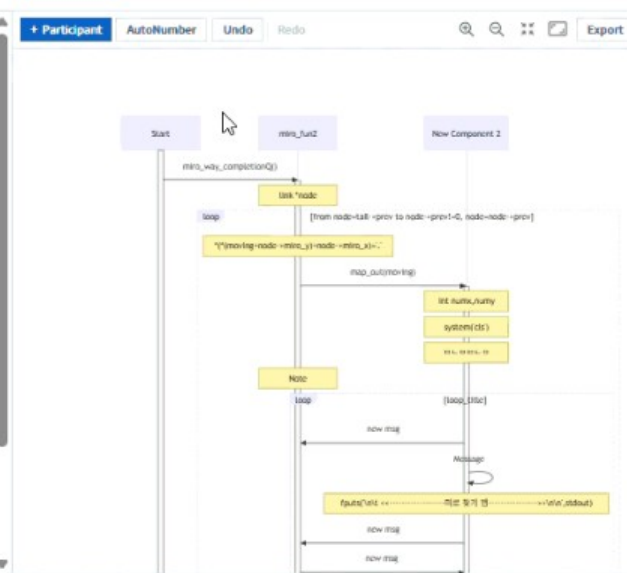


OLD - participate line 위주 편집 방식

Diagram Setting

Mermaid Code

```
sequenceDiagram
    participant Start
    participant miro_fun2
    participant New Component 2
    activate Start
    Start->>miro_fun2: miro_way_completionQ()
    activate miro_fun2
    note over miro_fun2: link * node;
    loop from node=tail->prev to node->prev=0, node=node->prev
        note over miro_fun2: *({moving: node->miro_y)+node->miro_x=-1;
        miro_fun2->>New Component 2: map_out(moving)
        activate New Component 2
        note over New Component 2: int numx,numy;
        note over New Component 2: system('cls');
        note over New Component 2: 0 1 0 0 1 0
        note over miro_fun2: Note
        loop loop_title
            New Component 2->>miro_fun2: new msg
            New Component 2->>New Component 2: Message
            note over New Component 2: fputs("\n\t <<-----대문 찾기 뽀-----
        >>\n\n';stdout);
            New Component 2->>miro_fun2: new msg
            miro_fun2->>New Component 2: new msg
        end
        loop from numy=0 to numy<16, increment by 1
            note over New Component 2: fputs("\t",stdout);
            loop from numx=0 to numx<16, increment by 1
                note over New Component 2: printf(" %c",{"(map+numy)+numx});
            end
        end
    end
```



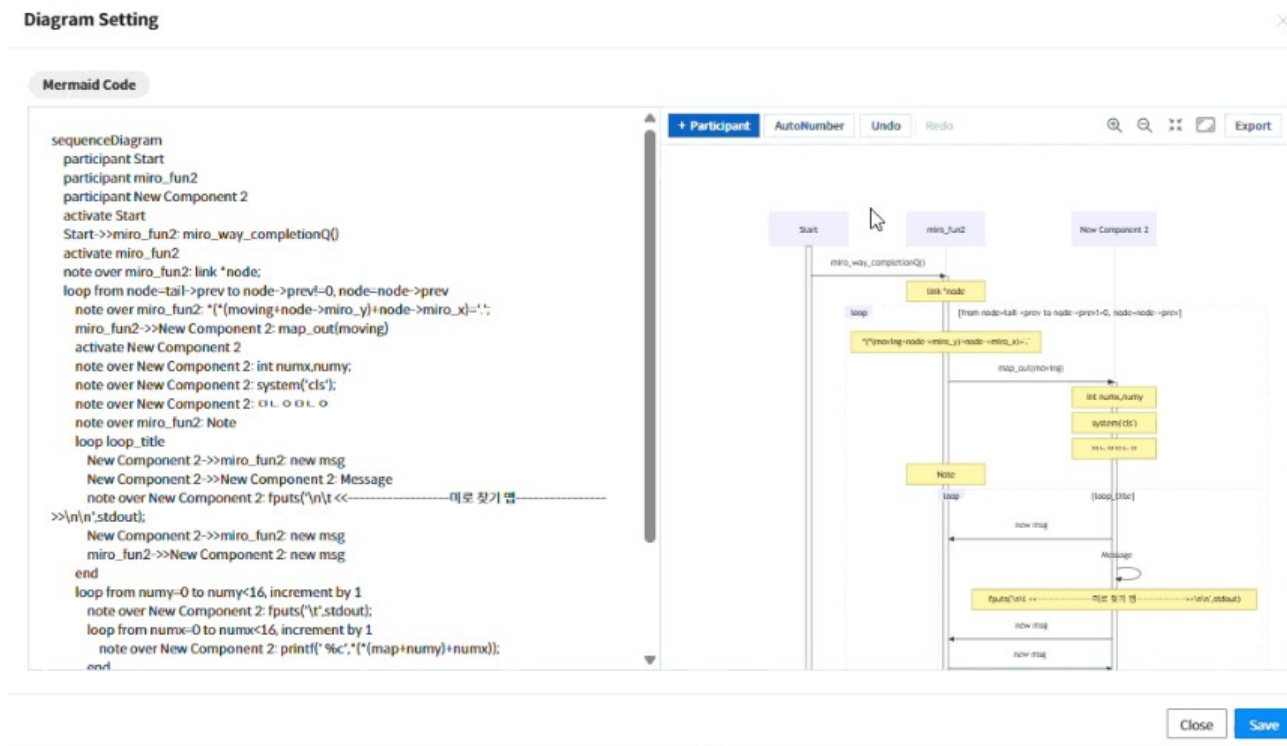
NEW - 새롭게 고안한 note/edge 객체 위주 편집 방식

01 Sequence Diagram (Dynamic)

SURESOFT

ALIRA - AI's Sequence Diagram

없으면 만들어보자



NEW - 새롭게 고안한 note/edge 객체 위주 편집 방식

(일반적인 시퀀스 다이어그램 형상에도 유효함을 확인. 사용자가 흐름을 따라가면서 해당 지점을 수정하기 더 용이한 방식)

좀 더 저희 용도에 맞는 방식을 고안해보았습니다.

사용자 시나리오

시퀀스 다이어그램을 백지에서부터 새롭게 만드는 사람보다는
AI가 생성한 결과물에서 흐름 따라가면서 고치는 경우가 대부분일거라고 판단.

그래서

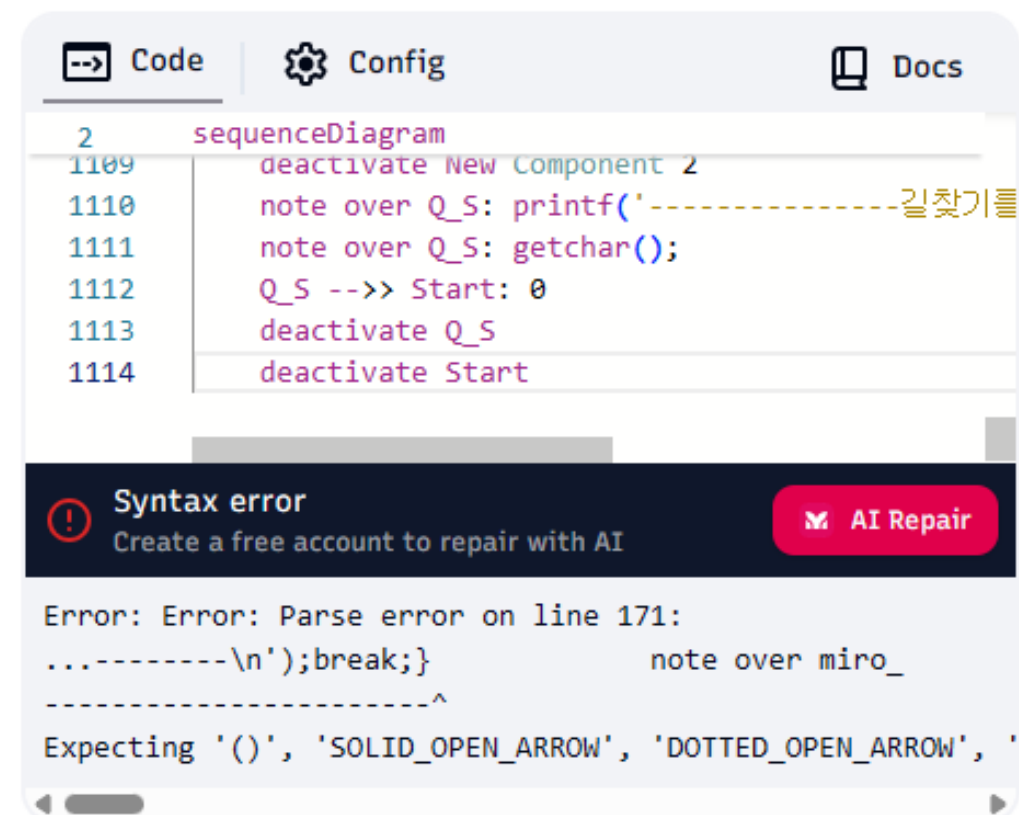
사용자가 흐름을 따라가다가 편집할 부분이 보이면
바로바로 세밀하고 손쉽게 수정이 가능하도록
edge와 note 객체를 기준으로 편집 버튼이 생기도록 변경

01 Sequence Diagram (Dynamic)

ALIRA - AI: Syntax Error

ALIRA가 생성하는 시퀀스 다이어그램 코드의 문법적 에러

≡ Mermaid Live Editor



ALIRA가 생성한 시퀀스 다이어그램 코드가 길어질수록
Syntax Error가 나는 코드를 받을 확률이 높아짐을 확인

SureFlow에서의 에러 말고도,
실제 Mermaid 공식에서도 에러나는 경우를 정리해봄.

- SureFlow에서 파싱 - 직렬화 단계에서 맞는 문법으로 교정이 가능하긴함
- 다만 여기서 고치는게 맞을지 아니면 ALIRA-AI의 개선을 기다리는게 나올지 고민

01 Sequence Diagram (Dynamic)

ALIRA - AI: Syntax Error

ALIRA가 생성하는 시퀀스 다이어그램 코드의 문법적 에러

1. 세미콜론 ; 문제

Mermaid

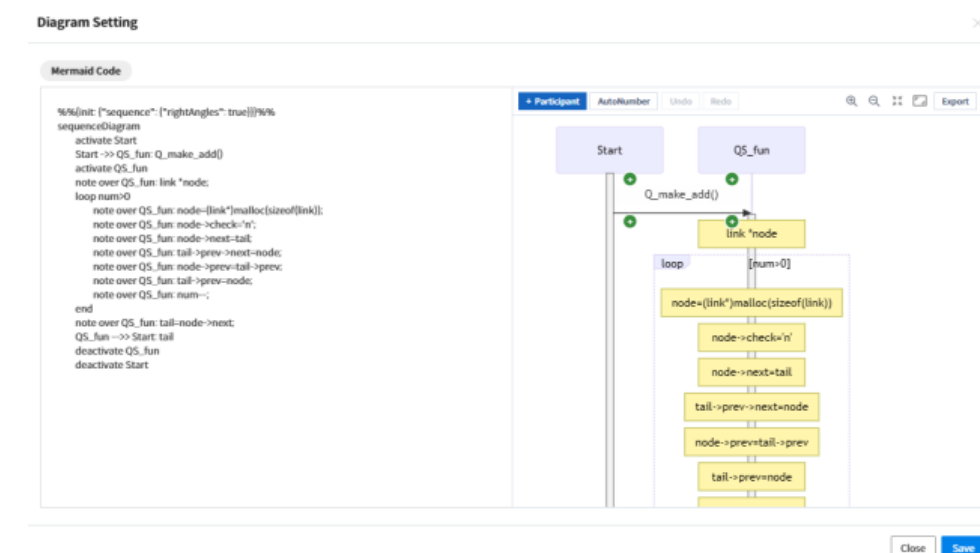
```
note over miro_fun2: link *node;  
note over New Component 2: int numx,numy;  
note over New Component 2: system('cls');  
note over New Component 2: printf(' %c',*(map+numy)+numx));
```

단순히 int x,y; 정도는 보통 통과

하지만 특히 아래처럼 한 줄에 C 문장이 여러 개 들어가면 더 위험 !!

```
note over miro_fun: printf('-----길을 찾기에 성공하였습니다.-----\n');break;}
```

애초에 Mermaid 코드상 ; 가 있어도 렌더링 할때는 없애고 그림.
; 를 표현하고 싶으면 #59;로 escape 해야 표현 가능.



결론 - escape #59를 코드에 추가하거나 ALIRA에서 생성할때 코드상 ; 는 빼고 생성하는건 어떨까 제안

01 Sequence Diagram (Dynamic)

ALIRA - AI: Syntax Error

ALIRA가 생성하는 시퀀스 다이어그램 코드의 문법적 에러

2. 공백 포함 participant 이름 문제

```
<> Mermaid  
  
miro_fun2 ->> New Component 2: map_out(moving)  
activate New Component 2  
note over New Component 2: int numx,numy;
```

Mermaid participant ID에 공백이 들어가면 파서가 New, Component, 2를 안정적으로 하나의 participant로 못 볼 수 있음.

보통 짧은 코드 상황에서는 괜찮지만,
시퀀스 다이어그램 코드가 길어지기 시작하면 Syntax Error 단골이 됨.

공백이 포함된 이름을 쓰고 싶다면, 안전한 방식은 alias를 쓰는 것.
아니면 단순히 participant 이름에서 공백 제거

```
participant NewComponent2 as "New Component 2"  
  
miro_fun2 ->> NewComponent2: map_out(moving)  
activate NewComponent2  
note over NewComponent2: int numx,numy  
deactivate NewComponent2
```

결론 - New Component 2에서 NewComponent2 or as "New Component 2"로 수정 제안

01 Sequence Diagram (Dynamic)

SURESOFT

ALIRA - AI: Syntax Error

ALIRA가 생성하는 시퀀스 다이어그램 코드의 문법적 에러

3. end 예약어 충돌 문제

실제 코드에 있는 `int end=0;; opt end == 2, else end==2, end=miro_check(...)`가 반복됨.

```
note over miro_fun: int end=0;
opt end == 2
    note over miro_fun: printf('-----길을 찾기에 성공하였습니다.-----\n');break;}
    note over miro_fun: break;
end
else end==2
```



```
opt "end == 2"
    note over miro_fun: break
end

alt "end == 2"
    note over miro_fun: path found
else "end != 2"
    note over miro_fun: continue
end
```

문제는 Mermaid에서 `end`는 `loop`, `opt`, `alt` 같은 블록을 닫는 키워드임.
`box` 문법이 `nested` 될수록 블록 종료 키워드 `end`와 충돌하는 경우 급증.

`end`를 " " 안에 넣는 방식이 제일 현실적

결론 - `end`라는 단어를 유지해야 하면 `alt/opt/loop` 라벨에서는 "`end == 2`"처럼 따옴표로 감싸는걸 제안

02 Component Diagram (Static)

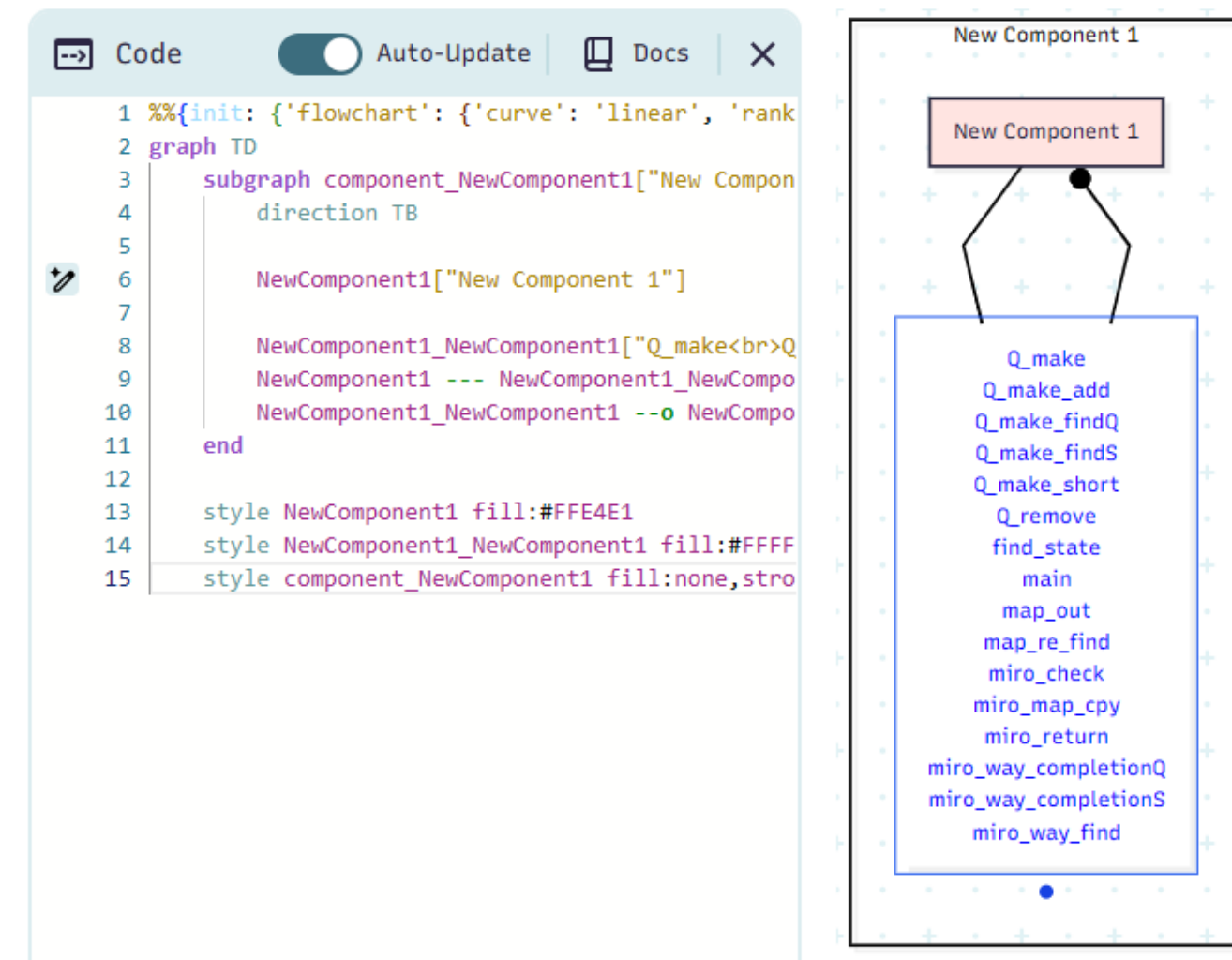
SAD - Static Diagram (Component Diagram)

ALIRA가 생성하는 Static Diagram

**New Component 1이라는 컴포넌트 안에
어떤 함수들이 들어 있는지를 보여주려는 다이어그램**

```
New Component 1
├─ Q_make
├─ Q_make_add
├─ Q_make_findQ
├─ Q_make_findS
├─ Q_make_short
├─ Q_remove
├─ find_state
├─ main
├─ map_out
├─ map_re_find
├─ miro_check
├─ miro_map_cpy
├─ miro_return
├─ miro_way_completionQ
├─ miro_way_completionS
└─ miro_way_find
```

구조적으로 보면,
이런 구조를 나타내려고 한 결과물로
이해를 했다.



02 Component Diagram (Static)

SAD - Static Diagram (Component Diagram)

ALIRA가 생성하는 Static Diagram

Mermaid Code

```
%%[init: {'flowchart': {'curve': 'linear', 'rankSpacing': 80, 'nodeSpacing': 50}}]%%  
graph TD  
  subgraph component_NewComponent1["New Component 1"]  
    direction TB  
  
    NewComponent1["New Component 1"]  
  
    NewComponent1_NewComponent1["Q_make<br>Q_make_add<br>Q_make_findQ<br>Q_mak  
e_findS<br>Q_make_short<br>Q_remove<br>find_state<br>main<br>map_out<br>map_re_fin  
d<br>miro_check<br>miro_map_cpy<br>miro_return<br>miro_way_completionQ<br>miro_wa  
y_completionS<br>miro_way_find"]  
    NewComponent1 --- NewComponent1_NewComponent1  
    NewComponent1_NewComponent1 --o NewComponent1  
  end  
  
style NewComponent1 fill:#FFE4E1  
style NewComponent1_NewComponent1 fill:#FFFFFF,stroke:none,color:#0000FF  
style component_NewComponent1 fill:none,stroke:#000
```

Static Diagram의 다이어그램 타입 Graph TD

graph는 Mermaid 초창기부터 쓰이던 기존/레거시 스타일 키워드,
flowchart는 나중에 더 명확하게 추가된 명시적 키워드.

결국 Static Diagram = Graph = Flowchart

이미 만들어진 기능이다!

02 Component Diagram (Static)

SAD - Static Diagram (Component Diagram)

Static Diagram = Graph = Flowchart

작업 목표 graph 문법으로 들어오는 static Diagram을 기존 flowchart 렌더링/편집 기반 위에서 다루되, 일반 flowchart와 동작을 섞지 않기 위해 static profile로 분리.

```
1 (function (global) {
2   'use strict';
3
4   var IDENT_SOURCE = '[A-Za-z_\\u3131-\\u079D][A-Za-z0-9_\\u3131-\\u079D]*';
5   var IDENT_RE = new RegExp('^(' + IDENT_SOURCE + ')$');
6
7   function markStatic(model, reason) {
8     if (!model) return;
9     model.profile = 'static';
10    if (!model.staticReasons) model.staticReasons = [];
11    if (reason && model.staticReasons.indexOf(reason) === -1) {
12      model.staticReasons.push(reason);
13    }
14  }
15
16  function decodeEscapedText(text) {
17    var out = '';
18    text = String(text || '');
19    for (var i = 0; i < text.length; i++) {
20      var ch = text.charAt(i);
21      if (ch === '\\') && i + 1 < text.length {
22        out += text.charAt(i + 1);
23        i++;
24      } else {
25        out += ch;
26      }
27    }
28    return out;
29  }
```

static-flowchart-parser.js

```
1 (function (global) {
2
3   function declarationsToString(declarations) {
4     var parts = [];
5     for (var i = 0; i < declarations.length; i++) {
6       if (!declarations[i].key) continue;
7       parts.push(declarations[i].key + ':' + declarations[i].value);
8     }
9     return parts.join(',');
10  }
11
12  function generateStyleLine(target, style, overrides) {
13    if (!target) return '';
14    var declarations = cloneDeclarations(style);
15    overrides = overrides || {};
16    for (var key in overrides) {
17      if (Object.prototype.hasOwnProperty.call(overrides, key)) {
18        upsertDeclaration(declarations, key, overrides[key]);
19      }
20    }
21    var body = declarationsToString(declarations);
22    return body ? 'style ' + target + ' : ' + body : '';
23  }
24
25  global.StaticFlowchartGenerator = {
26    generateDirectives: generateDirectives,
27  };
28}
```

static-flowchart-generator.js

담당 기능

- graph TD/TB/BT/LR/RL header 파싱
- %#{init: ...}% directive 파싱
- subgraph id["title"] 형태 파싱
- subgraph 내부 direction TB/LR/... 파싱

flowchart의 렌더링 자산을 그대로 재사용하면서
원본 보존이 필요한 부분만 profile: 'static'으로 격리.

02 Component Diagram (Static)

SAD - Static Diagram (Component Diagram)

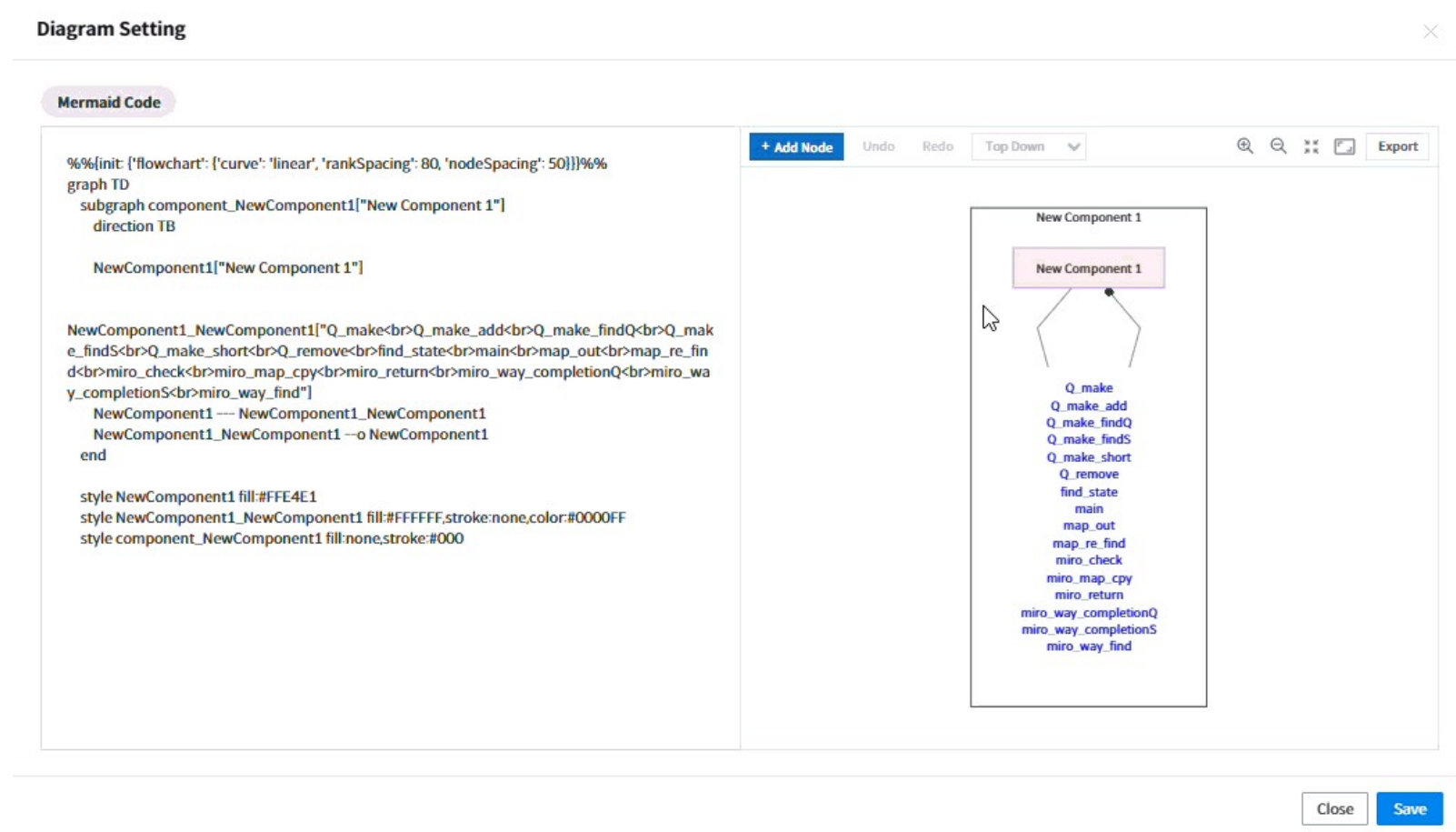
Static Diagram = Graph = Flowchart

VSPICE 내 임베드 완료

Sequence Diagram 임베드 할때

SadDiagramSettingModal 수정한 내용이 그대로 적용

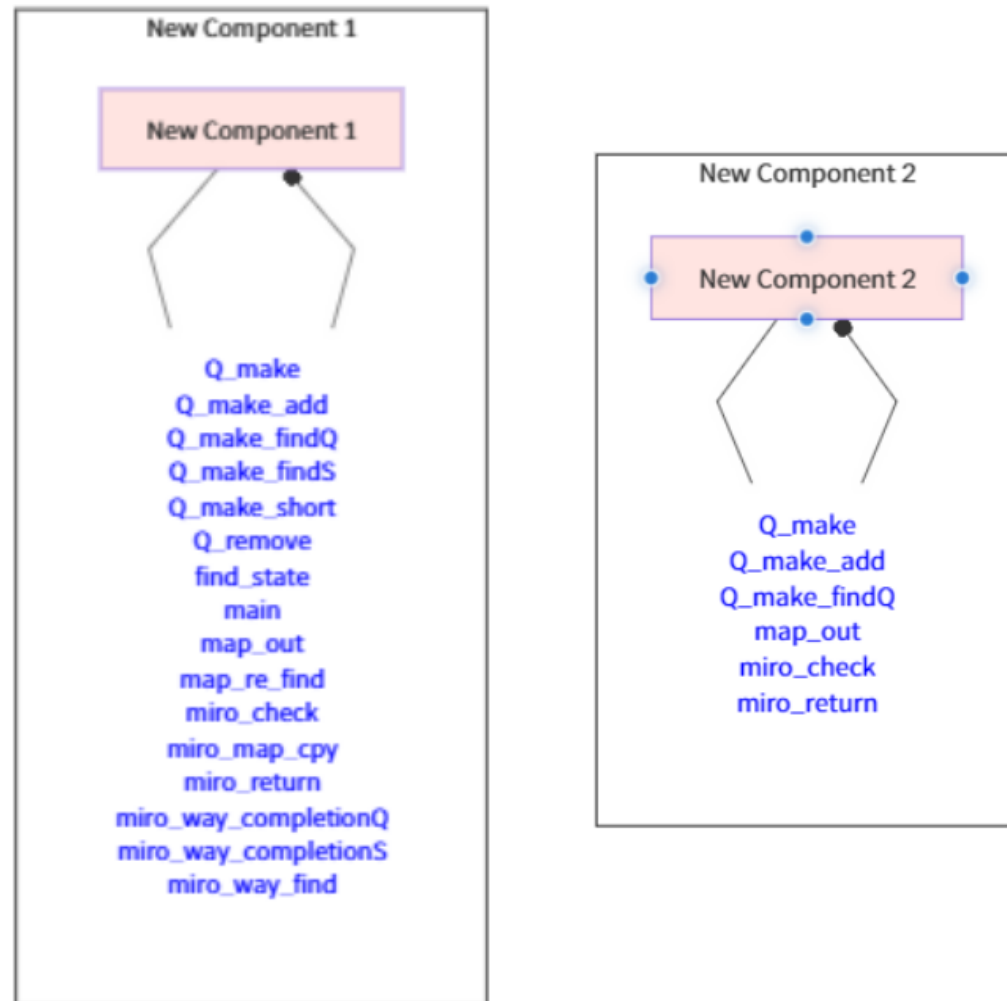
FlowChart와 동일한 로직으로 잘 작동함을 확인.



02 Component Diagram (Static)

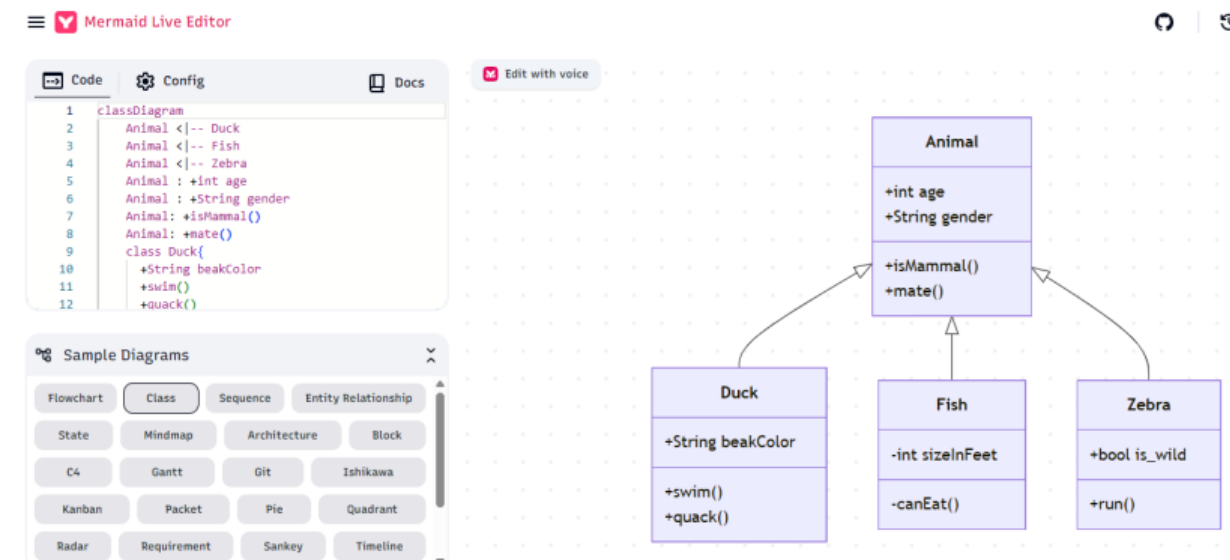
SAD - Static Diagram (Component Diagram)

ALIRA가 생성한 Static Diagram



이 Static / Component Diagram은 유효한가?

- C 코드 모듈별 함수 구성도
- 정적 함수 구성 다이어그램
- 컴포넌트 기반 함수 목록 다이어그램



Class Diagram? → 아직 정확히 똑같은 형상의 레퍼런스를 찾지 못했음.

감사합니다!